INTELLIGENT TRAFFIC SIGN DETECTION AND RECOGNITION USING
YOLOv8 AND CNN WITH DRIVER ALERT SYSTEM

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

ITA0506 - COMPUTER VISION

to the award of the degree of

BACHELOR OF TECHNOLOGY\BACHELOR OF ENGINEERING

IN

B.tech – AI & ML

Submitted by

Y. Tejanvesh Reddy (192425156)

Under the Supervision of

DR. R.C. JENI GRACIA

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**INTELLIGENT TRAFFIC SIGN DETECTION AND RECOGNITION USING YOLOv8 AND CNN WITH DRIVER ALERT SYSTEM**" has been carried out by **Y. Tejanvesh Reddy (192425156)** and under the supervision of Dr.R.C.Jeni Gracia and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Electronics and Communication Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	**COURSE COORDINATOR**
Dr.Carmel Mary Belinda M J,	**Dr.R.C.Jeni Gracia,**
Professor,	**Professor,**
SIMATS Engineering,	**SIMATS Engineering,**
SIMATS, Chennai – 602105.	**SIMATS, Chennai – 602105.**

DECLARATION

We, G.D.S.P. Santhosh Raja, **Y. Tejanvesh Reddy (192425156)**, of B.Tech-AI&ML, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Intelligent Traffic Sign Detection and Recognition using YOLOv8 and CNN with Driver Alert System" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Signature of the Students with Names

Y. Tejanvesh Reddy (192425156)

Place: Chennai
Date:

CERTIFICATE FROM THE SUPERVISOR

This is to certify that the Capstone Project entitled “Intelligent Traffic Sign Detection and Recognition using YOLOv8 and CNN with Driver Alert System” has been carried out by G.D.S.P. Santhosh Raja, **(192425156)** ,under the supervision of Dr. R.C. Jeni Gracia and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech-AI&ML programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

Supervisor: Dr. R.C. Jeni Gracia
Program Director: Dr. Sashi Rekha

Submitted for the Project Work Viva-Voce held on: ____________________

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INDIVIDUAL CONTRIBUTION STATEMENT

The project was completed as a team using a modular development approach. Responsibilities were distributed across design, development, testing, analysis, documentation and integration. Each member participated in review discussions and final system integration.

Student	**Primary Responsibility / Contribution**
G.D.S.P. Santhosh Raja	Video acquisition, preprocessing and integration support.
T.V. Sai Jagadeesh	YOLOv8 detection workflow, testing and report preparation.
C. Chandu	Multi-object tracking and temporal-analysis support.
Y. Sravan Kumar	ROI extraction, feature optimization and dataset preparation.
B. Rakesh	CNN classification workflow, testing and result analysis.
Y. Tejanvesh Reddy	Driver alert logic, application analysis and documentation support.

ABSTRACT

Road safety is significantly affected when drivers fail to notice or correctly interpret traffic signs, particularly at high speed, under poor visibility, in complex road environments, or during driver distraction. This project develops an intelligent computer-vision-based traffic sign detection and recognition system using YOLOv8 and CNN with a driver alert mechanism. The system captures real-time road video, performs adaptive preprocessing, detects traffic signs using YOLOv8, maintains sign identity across frames through tracking and temporal analysis, extracts and optimizes the region of interest, and classifies the sign using a convolutional neural network. The recognized sign is then converted into a visual or audio driver alert. The modular design addresses practical issues such as repeated detections, variable lighting, small sign regions and real-time processing. Evaluation is based on detection and classification behaviour, tracking consistency, response time, stability and alert reliability. The prototype demonstrates the feasibility of combining fast object detection, temporal tracking, ROI optimization and CNN-based recognition for driver assistance. The system is intended as an assistance mechanism rather than autonomous vehicle control, and its future development can include larger traffic-sign datasets, stronger adverse-weather robustness, transfer learning, embedded deployment, GPS integration and context-aware ADAS alerts.

Keywords: Traffic Sign Detection, YOLOv8, CNN, Computer Vision, Object Tracking, Driver Alert System

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation	Description
AI	Artificial Intelligence
ADAS	Advanced Driver Assistance System
CNN	Convolutional Neural Network
YOLO	You Only Look Once
ROI	Region of Interest
SORT	Simple Online and Realtime Tracking
FPS	Frames Per Second
OpenCV	Open Source Computer Vision Library

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Traffic signs are essential components of road transportation because they communicate speed limits, warnings, prohibitions, mandatory actions and road conditions. Drivers must continuously observe these signs while also monitoring vehicles, pedestrians and changing road conditions. Signs may be missed or incorrectly interpreted because of distraction, high vehicle speed, poor visibility, adverse weather, occlusion or complex road environments. Computer vision and deep learning provide a practical basis for an automated assistance system that can monitor road video, detect traffic signs and present useful information to the driver.

1.2 Need for the Project

A real-time traffic sign assistance system is needed because manual observation is not always consistent. Existing approaches can suffer from false detections, repeated detections across frames, difficulty with small or multiple signs, classification errors and processing delay. The project therefore integrates detection, tracking, temporal analysis, feature optimization, classification and alert generation rather than treating traffic sign recognition as a single isolated task.

1.3 Problem Statement

The engineering problem is to design a camera-based system capable of detecting, tracking and recognizing multiple predefined traffic signs from real-time road video and generating timely driver alerts under practical constraints such as variable lighting, motion blur, occlusion, sign-size variation, processing latency and limited computational resources. The design must balance detection speed, recognition reliability, tracking stability, usability and safety.

1.4 Project Aim

To develop an intelligent real-time traffic sign detection and recognition system using YOLOv8 and CNN, supported by multi-object tracking, ROI optimization and a driver alert mechanism.

1.5 Project Objectives

1. Develop a real-time video acquisition and adaptive preprocessing pipeline.
2. Use YOLOv8 for fast traffic sign localization and multiple-sign detection.
3. Track detected traffic signs across consecutive frames and use temporal information to reduce duplicate or unstable detections.
4. Extract and optimize traffic sign regions before classification.
5. Classify supported traffic signs using CNN or transfer-learning-based methods.
6. Generate understandable visual or audio alerts for the driver.
7. Evaluate the integrated system for detection behaviour, classification reliability, response time, tracking consistency and alert stability.

1.6 Scope

The project includes real-time camera/video input, preprocessing, YOLOv8-based detection, multi-object tracking, temporal analysis, ROI extraction, feature optimization, CNN-based classification and driver alerts. It excludes autonomous steering, braking, acceleration control and complete road-scene understanding. Performance can be affected by extreme lighting, severe weather, heavy occlusion, motion blur and signs outside the trained categories.

1.7 Expected Engineering Outcome

The expected outcome is a modular computer vision prototype that accepts road video, identifies supported traffic signs, maintains temporal consistency, classifies the detected sign and produces an immediate driver-facing alert. The architecture is intended to be maintainable and extensible for future ADAS or embedded deployment.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review considered traffic sign detection, recognition, object tracking and real-time computer vision approaches referenced in the project presentation and report. The comparison focuses on the technologies used, real-time suitability and the gaps that motivate the proposed integrated YOLOv8 + tracking + CNN + driver alert architecture.

2.2 Review of Existing Work

Traditional computer vision methods rely strongly on manually designed image features and may lose accuracy in complex scenes. CNN-based approaches improve recognition by learning visual features automatically. YOLO-family detectors provide fast end-to-end object localization and are better suited to real-time video. Tracking methods such as SORT/Deep SORT and Kalman filtering maintain object identity across frames, while ROI optimization reduces irrelevant background before classification.

2.3 Comparative Analysis

Existing Approach	**Advantages**	**Limitations**	**Relevance to Proposed Work**
Traditional image processing + HOG/SVM	Simple pipeline and interpretable features	Lower robustness in complex scenes; limited real-time accuracy	Motivates learned detection and classification features.
CNN-based recognition	Learns discriminative visual features automatically	Requires a reliable sign region and sufficient training data	Used for final sign classification.
YOLO-based detection	Fast detection and supports multiple objects	Small signs, adverse conditions and temporal duplication remain challenging	YOLOv8 selected as real-time detector.
YOLO + tracking	Maintains identity across video frames	Tracking can degrade under occlusion or missed detections	Temporal consistency added to the proposed design.
YOLOv8 + CNN + alerts	Combines fast localization, detailed classification and driver assistance	Requires integration, optimization and robust testing	Selected integrated architecture.

2.4 Research / Engineering Gap

The reviewed methods indicate a gap between isolated detection/classification models and a complete real-time driver-assistance workflow. The proposed project addresses this by combining adaptive preprocessing, YOLOv8 detection, multi-object tracking, temporal analysis, ROI optimization, CNN classification and alert generation in one modular pipeline.

2.5 Proposed Contribution

The proposed contribution is an integrated traffic sign understanding pipeline that uses YOLOv8 for rapid localization, temporal tracking for stable sign identity, ROI processing for cleaner classification input, CNN-based recognition for sign category prediction, and event-based driver alerts to convert recognition results into practical assistance.

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / Requirement	Type	Measurable / Verifiable Target
Driver - timely sign detection	Functional	Detect supported traffic signs from live/video frames and display a bounding box.
Driver - correct sign recognition	Functional	Classify supported detected sign regions and display the predicted class.
Driver - useful alert	Functional	Generate a visual and/or audio alert after a valid recognition event.
System - temporal consistency	Functional	Maintain sign identity across consecutive frames and reduce repeated alerts.
System - real-time operation	Non-Functional	Process continuous video with responsive output on the target prototype computer.
Developer - maintainability	Non-Functional	Use modular Python/OpenCV/YOLO/CNN components that can be independently tested and updated.

3.2 Constraints & Applicable Standards

Standard / Constraint	Requirement	How Applied
ISO 39001 - Road Traffic Safety Management	Road-safety-oriented system design	The system is positioned as driver assistance intended to improve sign awareness.
ISO/IEC/IEEE 12207 - Software Life Cycle Processes	Structured software development and maintenance	Requirements, modular design, implementation, testing and documentation are separated.
Human safety constraint	Alerts must not replace driver judgment	System outputs are advisory; autonomous control is outside scope.
Computational constraint	Real-time processing on practical hardware	YOLOv8 and modular image processing are selected for efficient video operation.
Environmental constraint	Variable illumination, blur, weather and occlusion	Adaptive preprocessing and future robustness improvements are considered.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Input	Live camera or road video	Video stream	High

Detection	YOLOv8 traffic sign localization	Bounding box + confidence	High
Tracking	Frame-to-frame sign identity	Track ID	High
Classification	CNN / transfer-learning sign class	Class label	High
Alert	Visual and/or audio warning	Event	High
Operation	Continuous responsive processing	Real-time prototype	High

3.4 Alternative Solutions & Evaluation

Alternative 1 uses traditional image processing with handcrafted features. Alternative 2 uses CNN classification on candidate sign regions. Alternative 3 uses the proposed YOLOv8 + tracking + CNN + driver alert architecture. Scores below are qualitative engineering ratings (1 = weak, 5 = strong) used for design selection rather than measured experimental results.

Criterion	**Weight**	**Traditional CV**	**CNN only**	**YOLOv8 + Tracking + CNN**
Real-time detection	0.25	3	2	5
Recognition capability	0.25	2	4	5
Multiple-sign handling	0.15	2	2	5
Temporal consistency	0.15	1	1	5
Extensibility	0.1	3	4	5
Driver-assistance integration	0.1	2	2	5

3.5 Selected Design & Detailed Design

The selected design is a six-module pipeline: (1) intelligent video acquisition and adaptive preprocessing, (2) real-time traffic sign detection using YOLOv8, (3) multi-object tracking and temporal analysis, (4) region extraction and feature optimization, (5) traffic sign classification using CNN, and (6) driver alert and decision support. The output of each stage forms the input to the next, while tracking information is used to stabilize detections and avoid unnecessary repeated alerts.

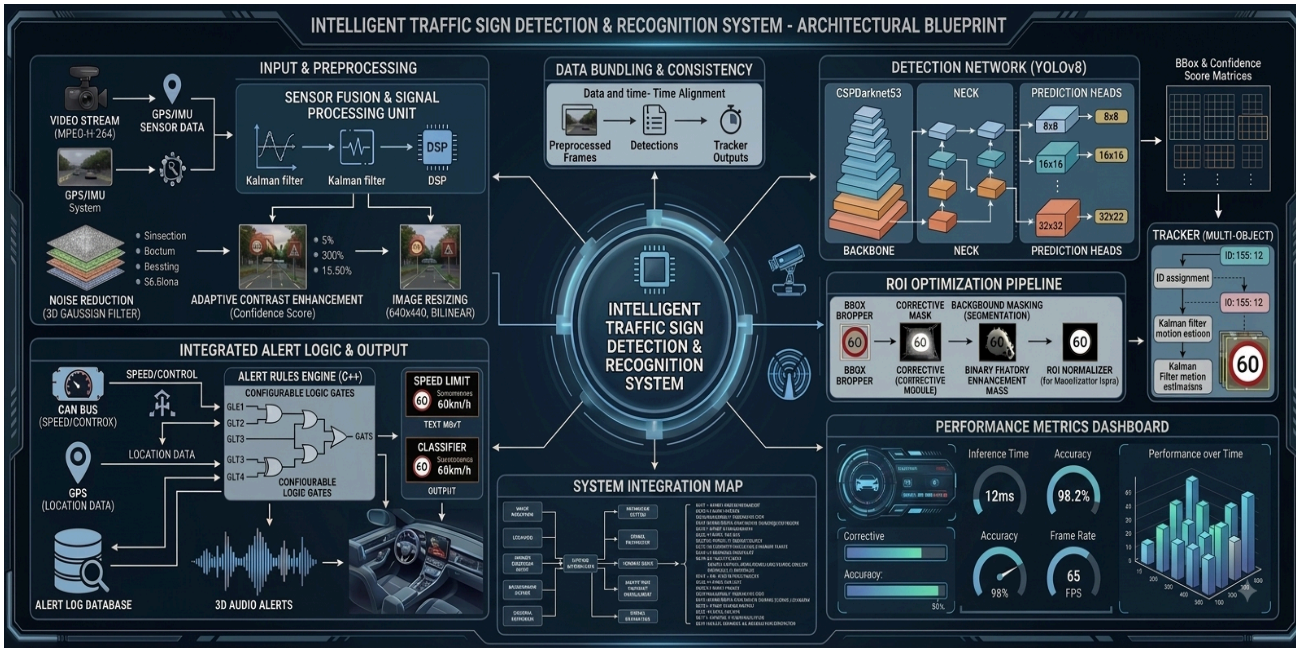

Figure 3.1 Proposed system architecture

3.6 Trade-off Analysis

YOLOv8 provides higher real-time suitability than a purely classification-first approach, but it introduces model-computation requirements. Tracking improves temporal stability but adds processing overhead and can fail under long occlusion. CNN classification provides detailed recognition but depends on good ROI quality and representative training data. The selected architecture accepts these computational costs because the combined pipeline provides stronger practical driver-assistance capability than any single stage alone.

3.7 Sustainability, Ethics, Safety, Risk & Security

- Sustainability: software-based processing can reuse standard cameras and computing platforms; future embedded optimization can reduce power consumption.
- Ethics: training and testing datasets should be used with appropriate licensing and acknowledgement; performance limitations should be reported accurately.
- Safety: alerts are advisory and must not be represented as a substitute for driver attention or legally required driving decisions.
- Risk: missed detections, false detections, misclassification, tracking loss and alert repetition can affect usability; mitigation includes preprocessing, confidence filtering, temporal consistency and testing under varied conditions.
- Security: future connected deployments should protect camera/video data, model files and communication interfaces from unauthorized access or tampering.

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows an iterative modular development approach. Individual modules are implemented and tested before complete integration. Real-time video is used to evaluate the behaviour of detection, tracking, ROI extraction, classification and alerts under varying backgrounds, sign sizes, distances and lighting conditions.

Resource / Tool	Purpose	Stage Used
Python	Core implementation and integration	All stages
OpenCV	Video capture, frame handling and preprocessing	Modules 1 and 4
YOLOv8	Traffic sign detection and localization	Module 2
SORT / Deep SORT / Kalman Filter	Object tracking and temporal consistency	Module 3
CNN / Transfer Learning	Traffic sign classification	Module 5
Text-to-Speech / GUI alert	Driver notification	Module 6
Camera / Webcam	Real-time video input	Testing and demonstration

4.2 Software Implementation & Modern Tools Used

4.2.1 Module 1 - Intelligent Video Acquisition & Adaptive Preprocessing

The system captures road video using a camera or webcam and separates it into frames. OpenCV is used for frame extraction, resizing, normalization, noise reduction and brightness/contrast adjustment. The objective is to provide consistent frames to the detection model under changing illumination.

4.2.2 Module 2 - Real-Time Traffic Sign Detection using YOLOv8

YOLOv8 processes the complete preprocessed frame, extracts learned features and predicts traffic sign bounding boxes with confidence scores. It can detect multiple traffic signs in the same frame, making it suitable for the project’s real-time detection requirement.

4.2.3 Module 3 - Multi-Object Tracking & Temporal Analysis

Detected signs are followed across consecutive frames. A unique tracking ID can be assigned to each sign using tracking techniques such as Kalman filtering and SORT/Deep SORT. Temporal information helps determine whether a sign is a continuation of a previous detection and can reduce repeated processing and alerts.

4.2.4 Module 4 - Region Extraction & Feature Optimization

The bounding-box region is cropped as the Region of Interest (ROI). Removing unnecessary background allows the classification model to focus on the sign. The ROI is resized, normalized and enhanced before classification.

4.2.5 Module 5 - Traffic Sign Classification using CNN

The optimized ROI is passed to a CNN, which learns edges, shapes, colours and patterns and predicts the traffic sign category. Transfer-learning models such as MobileNet or ResNet can be considered to reduce training time and improve performance when suitable training data is available.

4.2.6 Module 6 - Driver Alert & Decision Support System

The recognized sign is converted into an understandable warning. A visual message can be displayed, and an audio warning can be generated through text-to-speech. Tracking information is used to prevent the same sign from generating unnecessary repeated alerts.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Video acquisition	Run live/video input continuously	Frames captured and displayed without interruption.
Traffic sign detection	Present supported signs in video	Bounding box and confidence produced for visible supported signs.
Multiple-sign handling	Use frames containing more than one sign	Detector returns multiple sign regions where applicable.
Tracking consistency	Observe the same sign across consecutive frames	Track identity remains stable during normal visibility.
Classification	Classify extracted supported sign ROI	Predicted sign category is displayed.
Driver alert	Trigger alert after valid recognition	Visual/audio notification is generated without excessive repetition.
Robustness	Test lighting, distance, angle and partial occlusion variations	System remains operational and limitations are documented.

4.4 Results

The project report and presentation state that the integrated system can detect predefined traffic signs using YOLOv8, maintain detected signs across frames through tracking, classify extracted sign regions using CNN, and provide driver-facing alerts. The presentation also concludes that YOLOv8-based methods provide better real-time detection behaviour than traditional image-processing methods and that combining YOLOv8 with CNN, tracking and alerts improves practical usefulness. Numerical accuracy, FPS and latency values are not provided in the supplied source materials, so they are not invented in this report.

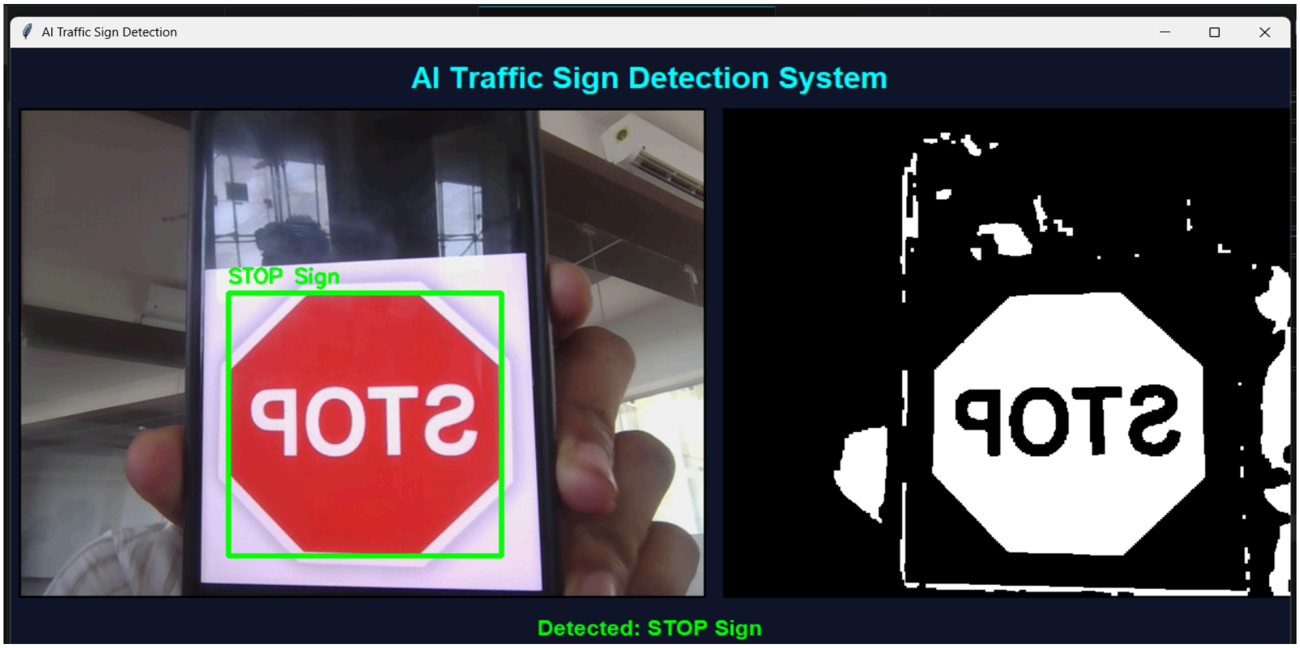

Figure 4.1 Implementation / traffic sign detection output from the project report

4.5 Data Analysis & Engineering Judgment

The main engineering observation is that a complete real-time solution benefits from separating localization and classification. YOLOv8 rapidly identifies candidate sign regions, while CNN classification focuses on the cropped ROI. Tracking adds temporal evidence that a sign persists across frames and supports alert suppression. The main remaining challenges are adverse lighting, motion blur, occlusion, visually similar signs and computational load when all modules operate together.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence from Project	Achievement
Real-time traffic sign detection	YOLOv8-based detection pipeline is defined and demonstrated in the project materials.	Fully
Traffic sign classification	CNN-based classification module is included in the architecture.	Fully
Multi-object tracking	Tracking and temporal analysis are included using SORT/Deep SORT/Kalman concepts.	Fully
ROI optimization	Cropping, resizing and normalization are defined.	Fully
Driver alert	Visual/audio alert mechanism is included.	Fully
Quantified performance validation	No complete measured accuracy/FPS/latency table is supplied.	Partially

4.7 Strengths & Limitations

Strengths

- Integrated end-to-end pipeline rather than isolated sign detection.
- YOLOv8 supports fast detection and multiple signs per frame.
- Tracking and temporal analysis improve continuity and can reduce duplicate alerts.
- ROI optimization supports focused CNN classification.
- Driver alert module converts model output into a practical assistance function.

Limitations

- Performance may decrease under heavy rain, fog, low light, strong sunlight, motion blur or severe occlusion.
- The system supports a predefined set of traffic sign categories.
- CNN performance depends on training-data diversity and ROI quality.
- Tracking may lose identity when detections are missed or signs are heavily occluded.
- The supplied project materials do not provide a complete quantitative benchmark for accuracy, FPS and latency.

4.8 Project Planning, Roles & Budget

Milestone	Main Activity
1	Problem definition and literature review
2	Architecture and module design
3	Video preprocessing and YOLOv8 detection
4	Tracking, ROI extraction and CNN classification
5	Driver alert integration
6	Testing, analysis, documentation and presentation

Student	Assigned Responsibility	Actual Contribution
G.D.S.P. Santhosh Raja	Module 1 / integration	Preprocessing and integration support
T.V. Sai Jagadeesh	Module 2 / testing	Detection workflow, testing and documentation
C. Chandu	Module 3	Tracking and temporal-analysis support
Y. Sravan Kumar	Module 4	ROI and feature-optimization support
B. Rakesh	Module 5	CNN classification and testing support
Y. Tejanvesh Reddy	Module 6	Driver alert and documentation support

Budget note: the supplied report describes a standard camera/webcam and a common personal computer as the prototype resources. No verified purchase-cost figures are provided, so a fabricated cost table has not been added.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project addresses the road-safety problem of missed or incorrectly interpreted traffic signs by developing an intelligent traffic sign detection and recognition architecture using YOLOv8 and CNN with a driver alert system. The proposed workflow combines adaptive preprocessing, fast sign localization, temporal tracking, ROI optimization, CNN-based classification and visual/audio alerts. The modular architecture provides a practical foundation for real-time driver assistance and supports future expansion. The supplied project evidence supports successful functional integration, while a complete numerical performance benchmark remains an area for further validation.

5.2 Future Work

- Expand the dataset to cover more regional and international traffic sign categories.
- Improve robustness under heavy rain, fog, low light, strong sunlight, motion blur and occlusion.
- Evaluate transfer-learning models such as MobileNet or ResNet for classification.
- Optimize YOLOv8 and tracking to reduce processing latency and support embedded deployment.
- Integrate GPS, digital maps and vehicle-speed information for context-aware alerts.
- Add multilingual voice alerts and configurable warning levels.
- Perform structured quantitative testing for accuracy, precision, recall, FPS, latency and alert reliability.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical understanding of computer vision pipelines for real-time video.
- YOLOv8-based object detection and CNN-based image classification.
- Object tracking, temporal consistency, ROI extraction and feature optimization.
- Integration of AI model outputs with a user-facing driver alert mechanism.

Engineering & Professional Skills Developed

- Python and OpenCV implementation skills.
- Modular system design, debugging, testing and parameter tuning.
- Problem solving under real-world image variations and processing constraints.
- Technical documentation, teamwork, communication and presentation.

Individual Reflection

The most difficult engineering challenge was maintaining reliable recognition when sign appearance changed because of distance, lighting, motion and occlusion. The modular design helped isolate detection, tracking and classification problems so they could be tested separately. A key design decision was to use YOLOv8 for localization and a separate CNN stage for detailed recognition, while tracking was used to improve temporal consistency. If the project were repeated, a larger labelled dataset and a formal benchmark of accuracy, FPS and latency would be planned from the beginning.

REFERENCES

1. R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed., Pearson Education, 2018.

2. R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022.

3. G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library, O'Reilly Media, 2008.

4. I. Goodfellow, Y. Bengio and A. Courville, Deep Learning, MIT Press, 2016.

5. C. M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006.

6. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," CVPR, 2016.

7. G. Jocher, A. Chaurasia and J. Qiu, Ultralytics YOLOv8, Ultralytics, 2023.

8. Y. LeCun, Y. Bengio and G. Hinton, "Deep Learning," Nature, vol. 521, pp. 436–444, 2015.

9. A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "Simple Online and Realtime Tracking," ICIP, 2016.

10. J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, "The German Traffic Sign Recognition Benchmark: A Multi-Class Classification Competition," IJCNN, 2011.

11. S. Houben, J. Stallkamp, J. Salmen, M. Schlipsing and C. Igel, "Detection of Traffic Signs in Real-World Images: The German Traffic Sign Detection Benchmark," IJCNN, 2013.

12. ISO 39001, Road Traffic Safety Management Systems - Requirements with Guidance for Use, International Organization for Standardization.

13. ISO/IEC/IEEE 12207, Systems and Software Engineering - Software Life Cycle Processes.

APPENDICES

```
import cv2

import numpy as np

import tkinter as tk
```

```
from PIL import Image, ImageTk

# ================================================================
# CAMERA SETUP
# ================================================================

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)

if not cap.isOpened():
    print("ERROR: Cannot open camera")

running = False

# ================================================================
# START CAMERA
# ================================================================

def start_camera():
    global running

    if not running:
        running = True
        update_frame()
```

```
# ============================================================
# STOP CAMERA
# ============================================================

def stop_camera():
    global running
    running = False


# ============================================================
# DETECT TRAFFIC SIGNS
# ============================================================

def detect_signs(frame):

    # Convert BGR to HSV
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # ========================================================
    # RED COLOR RANGE
    # ========================================================
```

```
  upper_red1
)
  cv2.countNonZero(roi_blue)
  / total_
# 2. PEDESTRIAN CROSSING
# Red Triangle
#
elif (

  # Label text
  cv2.putText(
    frame,
    sign_name,
    (x + 5, y - 8),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,

for sign in detected_signs:

  name = sign[0]
```

```
        if name not in sign_names:
            sign_names.append(name)

    detected_text = ", ".join(sign_names)
# ==========================================================

mask_img = ImageTk.PhotoImage(
    Image.fromarray(mask_rgb)
)

mask_label.config(image=mask_img)
mask_label.image = mask_img
if sign_crop is not None and sign_crop.size > 0:

    sign_crop = cv2.resize(
        sign_crop,
        (200, 200)
    )

    crop_rgb = cv2.cvtColor(
        sign_crop,
root.title("AI Traffic Sign Detection")
```

```
frame_main.pack()
video_label = tk.Label(
    frame_main,
    bd=2,
    relief="solid"
)

video_label.grid(
    row=0,
    column=0,
    padx=10
)
label_text = tk.Label(
    root,
    text="Detected: None",
    font=("Arial", 18, "bold"),
    fg="lime",
    bg="#0f172a"
)

label_text.pack(
    pady=20
)
```

```
btn_frame = tk.Frame(
    root,
    bg="#0f172a"
)
btn_frame.pack()
start_btn = tk.Button(
    btn_frame,
    text="START",
    width=15,
    height=2,
    command=start_camera,
    bg="green",

    row=0,
    column=1,
    padx=20
)
root.mainloop()
cap.release()
cv2.destroyAllWindows()
```